



[NOM DE L'ÉCOLE / UNIVERSITÉ]

FORMATION : [INTITULÉ DE LA FORMATION]

Rapport de Stage

Conception et développement d'un système d'historisation
au sein d'une application de gestion de financements de
formation professionnelle

Étudiant :

Prénom NOM

Tuteur entreprise (ISI.nc) : [Nom
du tuteur]

Tuteur académique : [Nom du
tuteur]

Stage réalisé chez ISI.nc, pour le compte du client FPG

Stage effectué du [date de début] au [date de fin]

Année [année scolaire]

Remerciements

Je tiens à remercier en tout premier lieu [Nom du tuteur entreprise], mon tuteur de stage au sein d'ISI.nc, pour son accompagnement tout au long de cette mission, la confiance qu'il m'a accordée sur un sujet technique exigeant, ainsi que pour sa disponibilité et ses conseils précieux.

Je remercie également l'ensemble de l'équipe projet FPG — Product Owner, développeurs et tech lead — pour leur accueil, leur pédagogie lors des échanges quotidiens et le temps qu'ils ont consacré à répondre à mes questions tout au long des sprints.

Mes remerciements vont aussi à l'ensemble du bureau ISI.nc pour son accueil chaleureux et l'environnement de travail stimulant qu'il m'a offert durant toute la durée du stage.

Enfin, je remercie [Nom du tuteur académique], mon tuteur pédagogique, pour son suivi et ses conseils dans la rédaction de ce rapport.

Table des matières

Table des figures

Chapitre 1

Introduction

1.1 Présentation personnelle

Actuellement étudiant en [intitulé de la formation], j’ai effectué un stage de [durée] au sein de l’entreprise ISI.nc, du [date de début] au [date de fin]. Mon parcours m’a progressivement orienté vers le développement web, et plus particulièrement vers le développement full stack, à la croisée du frontend et du backend. Ce stage représentait pour moi l’opportunité de mettre en pratique mes compétences sur un projet réel, de taille significative, au contact d’une équipe expérimentée.

J’ai choisi ce stage car [motivation personnelle : intérêt pour la mission proposée, réputation de l’entreprise, technologies utilisées, etc. — à personnaliser].

1.2 Présentation de l’entreprise d’accueil

ISI.nc est une entreprise spécialisée dans le développement de solutions logicielles sur mesure. Elle intervient auprès de divers clients pour concevoir, développer et maintenir des applications métier adaptées à leurs besoins spécifiques. L’entreprise s’appuie sur des méthodologies agiles et une expertise technique couvrant l’ensemble de la chaîne de développement, du frontend au backend, en passant par l’infrastructure.

1.3 Présentation du client FPG et du contexte métier

Le stage s’est déroulé dans le cadre d’une mission réalisée pour le compte du FPG (Fonds Paritaire de Gestion, ou équivalent selon le contexte réel), un organisme chargé de la gestion des demandes de financement de formation professionnelle déposées par des structures adhérentes (entreprises).

L’application développée accompagne l’ensemble du cycle de vie d’une demande de financement :

- le **dépôt** de la demande par la structure adhérente ;
- l'**instruction** du dossier par les équipes du FPG ;
- l'**engagement** financier associé à la demande ;
- le **règlement** des sommes dues.

Ce cycle mobilise plusieurs types d'entités métier interconnectées : stagiaires, montages financiers, tiers bénéficiaires, règlements, documents justificatifs, structures adhérentes et comptes collaborateurs. La complexité et la sensibilité de ces données — notamment financières — rendent la traçabilité des modifications apportées à ces dossiers particulièrement critique, ce qui constitue précisément le cœur de la mission qui m'a été confiée.

1.4 Annonce du plan

Ce rapport présente dans un premier temps le contexte de travail et la méthodologie adoptée, ainsi que la stack technique du projet. Il détaille ensuite en profondeur la mission principale du stage : la conception et le développement d'un système d'historisation des modifications. Les compétences acquises et les difficultés rencontrées sont ensuite discutées, avant de conclure sur le bilan global de cette expérience.

Chapitre 2

Méthodologie et environnement de travail

2.1 Méthodologie de travail

Le projet est mené selon la méthodologie Scrum, avec des sprints d'une durée de deux semaines. Cette organisation rythme l'ensemble du travail de l'équipe et structure la planification des développements.

Chaque journée est ponctuée de deux daily meetings :

- un premier avec le bureau ISI.nc, permettant de faire le point sur l'avancement individuel et les éventuels blocages internes ;
- un second avec l'équipe FPG (Product Owner, développeurs, tech lead), pour synchroniser l'avancement du sprint côté client.

En début de sprint, des séances de **Poker Planning** sont organisées pour estimer collectivement la complexité des tickets à traiter, favorisant une meilleure répartition de la charge de travail au sein de l'équipe.

Le suivi des tickets est assuré via l'outil **OpenProject** , qui centralise le backlog, les sprints en cours et l'état d'avancement de chaque tâche. La communication

interne quotidienne, quant à elle, s'effectue via **Mattermost** .

Les fonctionnalités sont décomposées en *User Stories*, détaillant les besoins fonctionnels du point de vue de l'utilisateur final. Ce découpage a par exemple été appliqué à la fonctionnalité d'historisation, cœur de ma mission (voir découpage détaillé en Annexe ??).

2.2 Stack technique

2.2.1 Développement

- **VS Code**  — IDE principal utilisé pour l'ensemble des développements, frontend comme backend.

2.2.2 Frontend

- **Angular**  — framework principal du frontend, associé à **PrimeNG** comme librairie de composants UI, offrant des composants riches (tableaux, formulaires, timeline, etc.) directement exploitables.
- **Gizmo**  — application constituant l'interface destinée aux structures adhérentes.

2.2.3 Backend

- **Python / Django** , associé à **Django REST Framework** pour l'exposition d'une API REST consommée par le frontend Angular.
- **django-simple-history** — bibliothèque permettant l'historisation automatique des modèles Django, point central de la mission qui m'a été confiée et détaillée au Chapitre ??.

2.2.4 Base de données

- **PostgreSQL**  — système de gestion de base de données relationnelle utilisé pour l'ensemble du projet.
- **DBeaver**  — client de base de données utilisé pour l'inspection et le débogage direct des tables, notamment des tables d'historique.

2.2.5 DevOps et infrastructure

- **GitLab**  — versionning du code, intégration et déploiement continu (CI/CD), gestion des *merge requests* et revues de code.

- **Linux**  — système d’exploitation utilisé comme environnement de développement.
- **Docker**  — conteneurisation des différents services du projet : base de données, stockage de fichiers (MinIO), backend Django.

2.3 Architecture générale du projet

L’architecture du projet suit un modèle classique d’application web découplée :

- un **frontend Angular** qui consomme une **API REST Django** exposée via Django REST Framework ;
- un **stockage de fichiers** (documents justificatifs, pièces jointes) assuré par **MinIO**, une solution de stockage objet compatible avec le protocole S3 ;
- une **génération de rapports PDF** déléguée à un serveur **JasperReports** externe, sollicité par le backend pour la production de documents officiels (attestations, décisions de financement, etc.).

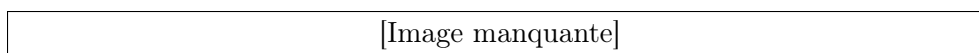


FIGURE 2.1 – Schéma de l’architecture générale du projet

Cette architecture découplée permet une séparation claire des responsabilités : le frontend se concentre sur l’expérience utilisateur, le backend sur la logique métier et l’accès aux données, tandis que les services externes (MinIO, JasperReports) prennent en charge des besoins spécialisés sans alourdir le cœur applicatif.

Chapitre 3

Mission principale : l’historisation des modifications

3.1 Contexte et besoin métier

Le FPG traite quotidiennement un grand nombre de dossiers de financement, impliquant des montants financiers conséquents et des données sensibles concernant les structures adhérentes et leurs collaborateurs. Dans ce contexte, la capacité à **tracer et afficher l’historique complet des modifications** apportées à un dossier constitue un besoin métier essentiel.

Concrètement, il s’agissait de permettre, pour chaque demande de financement, chaque structure adhérente et chaque compte collaborateur, de répondre à trois questions pour chaque modification apportée : **qui** a effectué la modification, **quand**, et **quelle était l’ancienne valeur par rapport à la nouvelle**. Cette fonctionnalité répond à des enjeux de :

- **traçabilité** — savoir précisément ce qui a été modifié et par qui à chaque étape du traitement d’un dossier ;
- **audit** — permettre un contrôle a posteriori du traitement des demandes de financement ;
- **résolution de litiges** — disposer de preuves fiables en cas de contestation d’une décision ou d’une donnée par une structure adhérente.

3.2 Choix techniques

3.2.1 django-simple-history

Le choix s’est porté sur la bibliothèque `django-simple-history`, qui génère automatiquement une table `_history` pour chaque modèle historisé. Chaque opération `INSERT`,

UPDATE ou DELETE effectuée sur un modèle historisé donne lieu à l'enregistrement d'une nouvelle ligne dans cette table, capturant l'intégralité de l'état de l'objet à cet instant, ainsi que l'utilisateur responsable de la modification et l'horodatage associé.

3.2.2 Le HistoryService

Afin d'éviter la duplication de logique entre les différentes entités historisées, j'ai construit un service dédié, le **HistoryService**, centralisant la récupération et la mise en forme de l'historique pour chaque type d'entité. Ce service constitue le point d'entrée unique utilisé par les différents endpoints d'API exposant de l'historique.

```

1 class HistoryService:
2     def get_history(self, entity, entity_id):
3         history_records = self._fetch_history(entity, entity_id)
4         resolved = self._resolve_references(history_records)
5         return self._format_for_frontend(resolved)

```

Listing 3.1 – Extrait simplifié du HistoryService

3.2.3 Système de snapshots

Une demande de financement regroupe de nombreuses entités liées (stagiaires, allocations budgétaires, tiers bénéficiaires, règlements). Comparer indépendamment l'historique de chacune de ces entités aurait rendu difficile la reconstitution d'un état cohérent du dossier à un instant donné.

J'ai donc mis en place un système de **snapshots**, matérialisé par le modèle **FundingRequestSnapshot** capturant à chaque sauvegarde un état complet et cohérent de l'ensemble des entités liées à une demande de financement. La comparaison entre deux snapshots successifs, via une méthode **diff_against**, permet d'obtenir de façon fiable l'ensemble des changements survenus entre deux états du dossier.

3.3 Défis techniques rencontrés

3.3.1 Modèles non rattachés au système de snapshot

Certains modèles — documents, rapports, comptes bancaires — ne sont pas directement rattachés au système de snapshot décrit précédemment. Pour ces entités, il a fallu développer une logique de comparaison indépendante, s'appuyant directement sur les tables d'historique générées par **simple_history**. Cette logique parcourt chronologiquement chaque enregistrement d'historique afin de détecter les transitions d'état pertinentes : création, suppression, ou changement de visibilité d'un document.

3.3.2 Relations Many-to-Many

L'historisation des relations Many-to-Many s'est révélée particulièrement délicate, notamment pour les labels associés aux documents. Le mécanisme d'historisation automatique des relations M2M fourni par `simple_history` s'est avéré insuffisant pour ce cas d'usage : il ne permettait pas de restituer de façon fiable et lisible les ajouts et retraits de labels au fil du temps.

La solution retenue a consisté à créer un modèle intermédiaire explicite, `DocumentLabel`, disposant de son propre historique. Cette approche, bien que plus verbeuse à mettre en place, offre un contrôle total sur la manière dont les associations sont historisées et restituées.

3.3.3 Résolution des données de référence

Les changements bruts capturés par `django-simple-history` contiennent des identifiants techniques (clés étrangères) plutôt que des libellés métier directement compréhensibles par un utilisateur final. Un changement de secteur d'activité, par exemple, n'apparaît dans l'historique brut que sous la forme d'un identifiant numérique.

J'ai donc développé une logique de résolution systématique, transformant chaque identifiant en son libellé métier correspondant (secteurs d'activité, codes NAF, codes ROME, catégories socio-professionnelles, pays, etc.) avant transmission des données au frontend, garantissant un affichage directement exploitable par les utilisateurs.

3.3.4 Cohérence entre données dupliquées

Au cours du développement, j'ai identifié un bug affectant certains champs relatifs aux employés (fonction, catégorie socio-professionnelle, niveau de formation) : ces champs étaient accidentellement réinitialisés à `NULL` lors de mises à jour partielles ne les concernant pas. La cause provenait d'une incohérence entre la structure du payload envoyé par le frontend et celle attendue côté backend. Ce bug, bien que sans lien direct avec l'historisation, a été détecté et corrigé grâce à l'analyse des tables d'historique, qui révélaient clairement les réinitialisations non désirées.

3.3.5 Fusion d'événements liés

Une même action utilisateur peut déclencher plusieurs sauvegardes techniques distinctes côté backend (par exemple lors de la mise à jour en cascade de plusieurs entités liées). Sans traitement particulier, l'historique présenté à l'utilisateur se serait retrouvé fragmenté en de multiples entrées correspondant en réalité à une seule action métier.

J'ai donc mis en place une logique de regroupement des événements survenus dans une fenêtre de temps rapprochée, afin de présenter une vue cohérente et lisible de l'historique,

alignée avec la perception qu'a l'utilisateur de ses propres actions.

3.4 Développement frontend

3.4.1 Composant réutilisable `tab-changes`

Côté frontend, j'ai conçu un composant Angular réutilisable, `tab-changes`, chargé d'afficher l'historique des modifications sous forme de timeline chronologique, à l'aide du composant PrimeNG `p-timeline`.

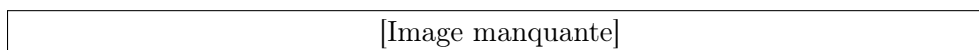


FIGURE 3.1 – Aperçu du composant d'historique final

3.4.2 Regroupement thématique et labels

Les modifications sont regroupées par thématique (formation, montage financier, stagiaires, tiers bénéficiaires, règlements, documents), avec un système de correspondance associant chaque champ technique à son libellé métier affiché à l'utilisateur.

3.4.3 Distinction visuelle des types de modification

Un code couleur permet de distinguer immédiatement les ajouts, les modifications et les suppressions, facilitant la lecture rapide de l'historique par les utilisateurs métier.

3.4.4 Gestion des cas particuliers d'affichage

Une attention particulière a été portée à la gestion de cas d'affichage spécifiques : valeurs numériques, dates, booléens, ainsi que des objets composites tels qu'un RIB accompagné de son justificatif, ou un organisme de formation disposant de plusieurs attributs à afficher conjointement.

3.4.5 Réutilisabilité de l'architecture

Le pattern d'historique ainsi développé a ensuite été répliqué sur plusieurs entités du système : demandes de financement, structures adhérentes, comptes collaborateurs et employés. Cette réplication a démontré la solidité et la réutilisabilité de l'architecture mise en place, tant côté backend (`HistoryService`, snapshots) que côté frontend (composant `tab-changes`).

3.5 Méthodologie de débogage

Face aux anomalies rencontrées durant le développement, j'ai adopté une démarche de diagnostic systématique, consistant à vérifier l'ensemble de la chaîne technique afin d'isoler précisément la source d'un dysfonctionnement :

1. vérification des données au niveau de la **base de données** elle-même, à l'aide de DBever ;
2. inspection du **service backend** concerné (`HistoryService` ou logique associée) ;
3. contrôle du **serializer DRF** responsable de la mise en forme des données exposées par l'API ;
4. validation du contenu exact des échanges via l'**onglet réseau** des outils de développement du navigateur ;
5. inspection du **composant frontend** recevant et affichant les données.

Le **shell Django interactif** a également constitué un outil précieux, me permettant d'inspecter directement l'état des données et des tables d'historique en base, sans passer par l'interface graphique.

Chapitre 4

Compétences et apprentissages

4.1 Compétences techniques acquises

- Approfondissement de **Django REST Framework**, notamment la conception d'API REST et de serializers complexes.
- Maîtrise renforcée de **PostgreSQL**, en particulier pour l'analyse de schémas de données historisées.
- Montée en compétence sur **Angular** / **TypeScript** avancé, notamment la conception de composants réutilisables.
- Pratique quotidienne de **Git** / **GitLab** en contexte d'équipe (branches, merge requests, revues de code).
- Utilisation de la **conteneurisation Docker** au quotidien pour le lancement de l'environnement de développement.
- Renforcement de mes capacités de **débogage full stack**, de la base de données jusqu'à l'interface utilisateur.

4.2 Compétences méthodologiques

- Pratique concrète de la méthodologie **Scrum** au sein d'une équipe projet établie.
- Participation au découpage de fonctionnalités en **User Stories**.
- Participation à des séances d'**estimation collaborative** (Poker Planning).

4.3 Compétences transversales

- Développement de ma capacité à **communiquer avec une équipe produit externe** (FPG), aux attentes et au vocabulaire métier propres.

- Gain en **autonomie** dans la résolution de bugs complexes, impliquant plusieurs couches applicatives.

Chapitre 5

Difficultés rencontrées et solutions apportées

Au-delà des difficultés techniques déjà détaillées au Chapitre ??, plusieurs difficultés d'un autre ordre ont marqué ce stage.

5.1 Montée en compétence sur un existant complexe

Le projet FPG constituait, au démarrage du stage, une base de code existante et déjà conséquente. La compréhension de son organisation, de ses conventions internes et de la logique métier sous-jacente a nécessité un temps d'adaptation important avant de pouvoir intervenir efficacement, en particulier sur un sujet aussi transverse que l'historisation, qui touche de nombreuses entités du système.

5.2 Coordination entre deux équipes

Le travail au sein d'un dispositif impliquant deux équipes distinctes — le bureau ISI.nc et l'équipe produit FPG — a nécessité une adaptation constante en termes de communication : reformuler les problématiques techniques pour un public parfois moins technique côté FPG, et à l'inverse, faire remonter les contraintes métier auprès de l'équipe ISI.nc.

5.3 Environnement de développement à dépendances externes

L'environnement de développement reposait sur plusieurs services externes conteneurisés via Docker (base de données, MinIO) ainsi que sur un service tiers, JasperReports, pour la génération de rapports PDF. La gestion de ces dépendances, parfois source de

dysfonctionnements difficiles à diagnostiquer (services non démarrés, versions incompatibles), a constitué un apprentissage à part entière sur la robustesse d'un environnement de développement full stack.

Conclusion

Ce stage au sein d'ISI.nc, réalisé pour le compte du client FPG, m'a permis de mettre en pratique l'ensemble de mes compétences en développement full stack sur un projet réel, complexe et exigeant. La mission qui m'a été confiée — la conception et le développement d'un système d'historisation des modifications — s'est révélée particulièrement formatrice, tant sur le plan technique (architecture backend, conception de composants frontend réutilisables, débogage transverse) que sur le plan méthodologique (Scrum, collaboration avec une équipe produit externe).

Au regard des objectifs pédagogiques fixés en début de stage, je considère que les compétences visées ont été atteintes, voire dépassées sur certains aspects techniques que je ne maîtrisais pas initialement, comme l'historisation avancée avec `django-simple-history`.

Cette expérience conforte mon projet professionnel de poursuivre dans le développement full stack, et m'a permis de mesurer concrètement l'importance de la rigueur, de la communication et de la démarche de débogage méthodique dans un contexte professionnel.

Sur le plan des perspectives, la fonctionnalité d'historisation développée pourrait être enrichie par des fonctionnalités de recherche et de filtrage avancées dans l'historique, ou encore par des exports dédiés à des fins d'audit, ouvrant la voie à de futures évolutions pour l'équipe qui poursuivra ce projet.

Bibliographie / Webographie

- Documentation officielle Django — <https://docs.djangoproject.com/>
- Documentation officielle Django REST Framework — <https://www.django-rest-framework.org/>
- Documentation django-simple-history — <https://django-simple-history.readthedocs.io/>
- Documentation officielle Angular — <https://angular.dev/>
- Documentation officielle PrimeNG — <https://primeng.org/>
- Documentation officielle PostgreSQL — <https://www.postgresql.org/docs/>
- Documentation GitLab — <https://docs.gitlab.com/>

Annexe A

Annexes

A.1 Découpage en User Stories de la fonctionnalité d'historisation

[À compléter à partir du document de découpage en User Stories fourni.]

A.2 Extraits de code significatifs

```
1 class DocumentLabel(models.Model):  
2     document = models.ForeignKey(Document, on_delete=models.CASCADE  
3     )  
4     label = models.ForeignKey(Label, on_delete=models.CASCADE)  
5     history = HistoricalRecords()
```

Listing A.1 – Exemple : modèle intermédiaire pour l'historisation M2M

A.3 Captures d'écran de l'interface d'historique

[Image manquante]

FIGURE A.1 – Vue détaillée d'une entrée d'historique